

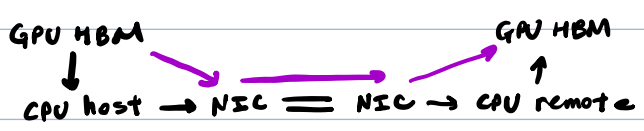
# Non-collective / fine-grained P2P Comms needed for:

- Data-dependent communication
  - CPU Coordination too slow
  - Comm. for subset of threads
- e.g. stencil halo exchange  
irregular graph Comms.  
sparse / EP workloads  
tightly-coupled kernels

## Terms

### - GPUDirect RDMA:

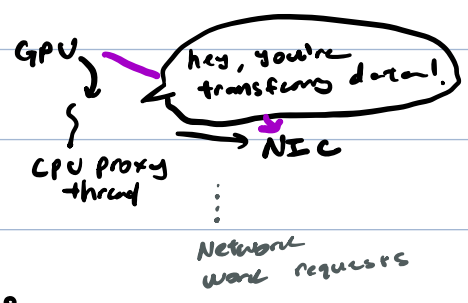
↳ Hardware capability



Data path

### - GDAKI

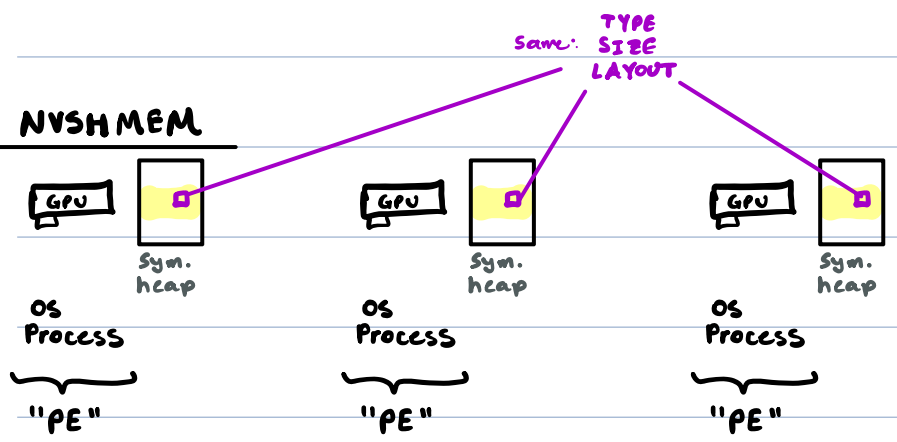
- ↳ Hardware capability
- ↳ Exposes network ctrl regs. directly to the SMs:
- no host proxy thread needed



Control path

↳ IBGDA: GDAKI, but the NICs/network in question use IB

## NVSHMEM



## The NVSHMEM API

- nvshmem\_ → Part of openSHMEM
- nvshmemx\_ → extension w/ stream-ordering & warp block variants
- nvshmemi\_ → internal routines: NOT to be called by users (underlies public APIs)

## Initialization [HOST]

```
nvshmem_init()
nvshmem_finalize()

nvshmem_my_pe()
nvshmem_n_pes()
```

## Memory [HOST]

```
nvshmem_malloc(size_t size)
nvshmem_free(void* ptr)
nvshmem_align
```

## RMA Operations (remote memory access)

### PUBLIC API

nvshmem-

**DEVICE** - Call API in a kernel

**HOST** - Like a normal memory outside a kernel btwn ANY two GPUS

|                                  |                 |                                                                      |
|----------------------------------|-----------------|----------------------------------------------------------------------|
| Typed<br>- <type>                | Bulk-Contiguous | -put TARGET<br>-get (DST, SRC, CNT, TARGET PE)                       |
|                                  | scalar          | -p TARGET<br>-g (DST, value, TARGET PE)                              |
|                                  | Strided         | -iput TARGET<br>-iget (DST, SRC, STRIDE, SRC STRIDE, CNT, TARGET PE) |
| Size-Specialized<br>- ... <SIZE> | Bulk-Contiguous | -put <SIZE><br>-get <SIZE>                                           |
|                                  | scalar          | -p <SIZE><br>-g <SIZE>                                               |
|                                  | Strided         | -iput <SIZE><br>-iget <SIZE>                                         |
| Untyped                          | Bulk-Contiguous | -putmem TARGET # TARGET<br>-getmem (DST, SRC, BYTES, PE)             |

## Warp/block - Scoped Modifiers

nvshmemx-

### DEVICE

Thread scope nvshmem - put32  
Warp scope nvshmemx - put32 -warp  
Block scope nvshmemx - put32 -block

Faster! But all participating threads must be synced & all calling at once!

### HOST

Synchronous nvshmem - put32  
Stream-ordered nvshmemx - put32 -on-stream (TARGET DST, SRC, size, TARGET PE, stream)

Public APIs are a wrapper around:

## PRIVATE API (nvshmemi-)

Fast

(P2P NVLink)

Slow

(IBGDA  
GOAKI / Proxy)

Device

Host

Bulk-host

(-prepare-and-post-rma)

Router!

(-p2p-rma-optimized)

cudaMemcpy Device to Device

Scalar  
(-P/-G)

Bulk-device  
(-put-threadgroup)

tid = threadIdx.x  
for (i = tid; i < cnt; i += 32):  
dst[i] = src[i]

offset  
= dst - local  
addr heap  
base

dst\_remote  
= Peer  
heap [Pe] + offset  
base

\*dst\_remote = value

Device

Host

Routers!

Router!

Scalar  
Bulk-device  
Signal

nvshmemi-transfer-rma-{p/g}  
nvshmemi-transfer-rma{-nbi}  
nvshmemi-transfer-put-signal  
nvshmemi-transfer-amd-\*

nvshmemi-prepare-and-post-rma

IBGDA  
GOAKI

nvshmemi-ibgda-rma-\*

↳ construct IB  
work request  
↳ GPU directly posts  
to NIC registers

Proxy  
fallback

↳ Create proxy req. struct  
{dst, src, bytes, ...}

↳ write to pinned  
proxy buffer

CPU proxy thread executes:

while true:

req = read-host-pinned-proxy-buffer()

if req.op == scalar:

nvshmemi-proxy-rma-{p/g} (← req →)

Synchronous  
nvshmemi-process-  
multisend-rma

Asynchronous  
(on-stream)  
nvshmemi-proxy-  
rma-launcher

calls libibverbs or  
whatever fabric API

↓  
Async version enqueues  
onto a CUDA stream

## MEMORY ORDERING

### nvshmem-fence

↳ Nonblocking

↳ nvshmem-put (big payload, 2)  
nvshmem-fence  
nvshmem-put (small signal, 2)  
[GPU does other math]

guarantees payload arrives BEFORE signal

↳ MUST be same dst PE: else ordering invalidated

↳ nvshmem-put (big payload, 2)  
nvshmem-fence  
nvshmem-put (small signal, 1)  
[GPU does other math]

NO ordering guarantee

### nvshmem-quiet

↳ Blocking & applies to ALL PEs

↳ EVERY line prior is complete & data visible

### nvshmemx-quiet-on-stream

↳ GPU pauses stream until ALL prior network ops have cleared.

## SYNCING

mostly called in kernels

if host, use in streams  
(polling GPU memory)

void nvshmem-<TYPE>-wait-until (ivar, cmp, cmp-value)  
↳ spin until ivar cmp cmp-value true!  
↳ BLOCKING

Sym. Var  
on local  
PE

GT  
EQ  
NE

Some value

int nvshmem-<TYPE>-test (ivar, cmp, cmp-value)  
↳ whether ivar cmp cmp-value true/false  
↳ NON-BLOCKING

## COLLECTIVES

Call from both  
**HOST & DEVICE**

Barrier  
Sync  
Broadcast

AlltoAll  
Fcollect (Allgather)  
Reduce  
Reduce Scatter

## ATOMICS

↳ if N threads all try to run these fns, we guarantee they sequentialize

↳ {read modify write} 1 unbreakable step

↳ "fine-grained coordination on specific memory locations"

nvshmem-<TYPE>-atomic-<OP>  
add

no effect on the existing value

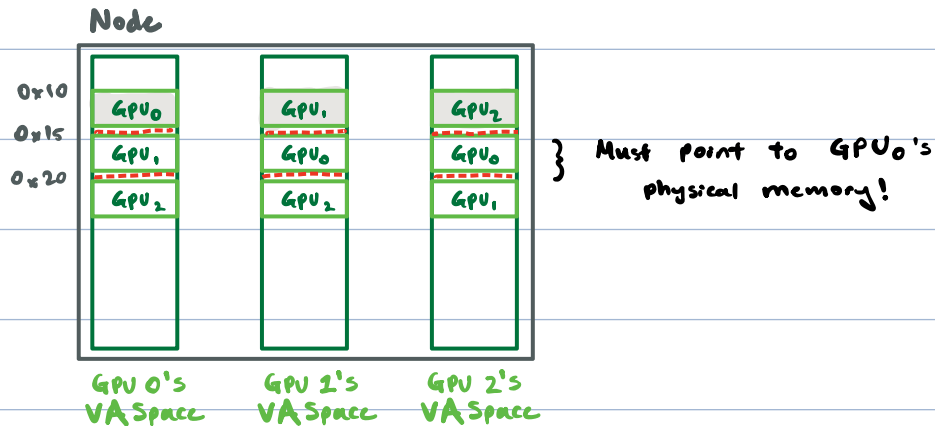
nvshmem-<TYPE>-atomic-fetch-<OP>

old = nvshmem-int-atomic-fetch-add  
Modify remote AND return old value to caller

<OP> can be compare-and-swap, swap, & fetch  
↳ swap only if condition is met

# Low Level Memory Setup

- Multi-gpu == Kernels accessing HBM on peer devices
- == need a mapping!



## ① CUDA UVA: one process

- + just on VASpace - no issues!
- Torchrun assumes 1 GPU per 1 process

```

cudaSetDevice(0)
cudaMalloc(&gpu0_ptr, size)

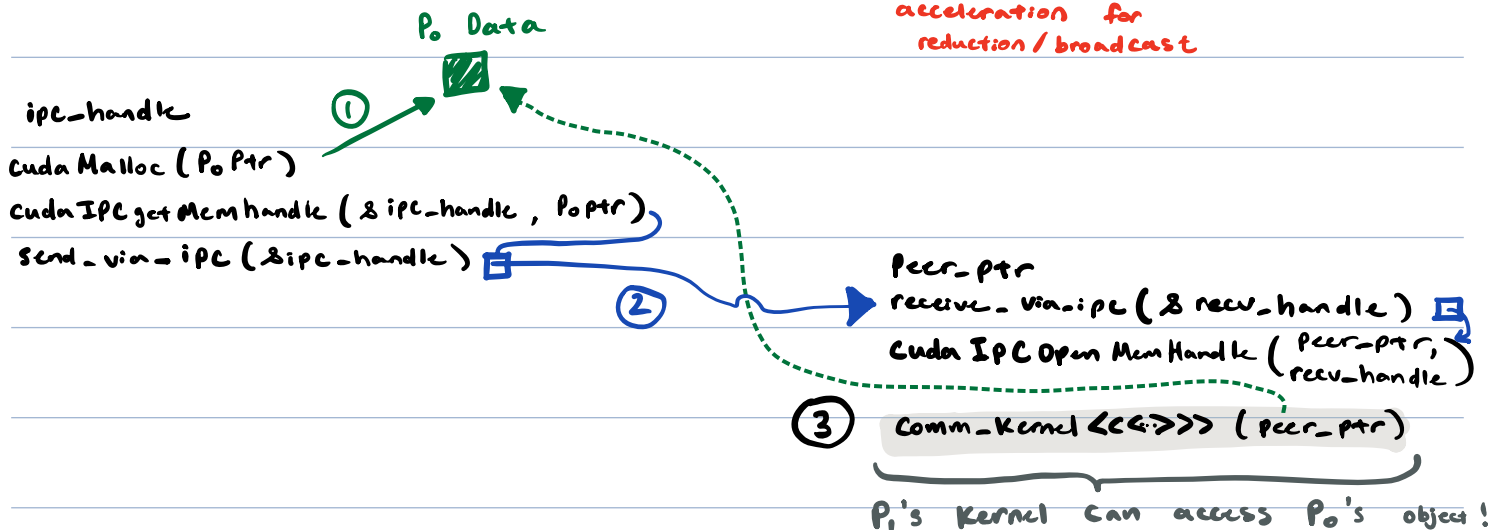
cudaSetDevice(1)
cudaDeviceEnablePeerAccess(0,0)

writeToPeerKernel<<<...>>>()
    
```

## ② CUDA IPCHandles : >1 process

- + works on preallocated cudaMalloc'd tensors

- Cannot use NVLS acceleration for reduction / broadcast



### ③ VMM

Lets you program page tables directly!

- Need custom tensor class  
(allocated & deallocated w/ VMM logic)

+ NVLS acceleration!

cuMem Address Reserve

- claim block of VAs

cuMem Create

- claim block of Silicon

cuMem Map

- update page table: {VA → Silicon}

cuMemSet Access

- WHO can see this mapping

# Symmetric Memory Backend

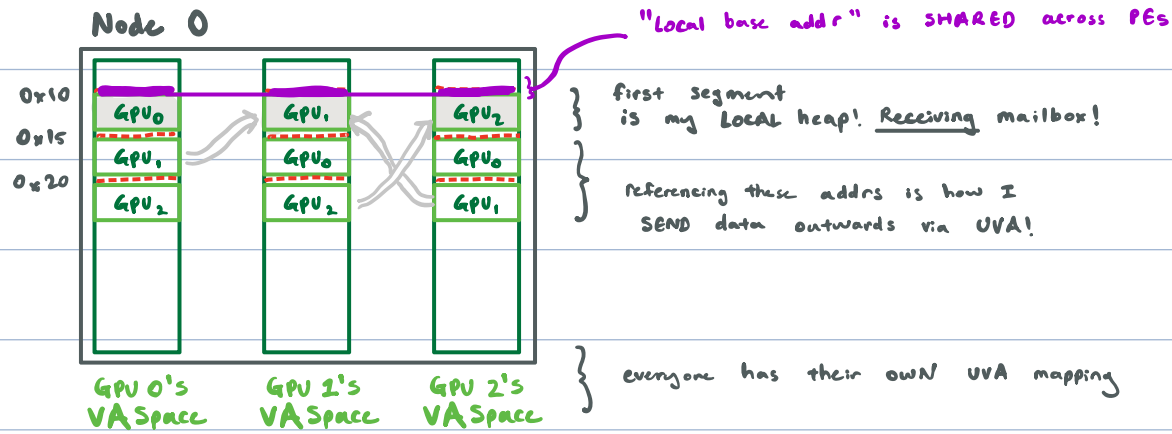
nvshmem\_init() - Sets up the PGAs!  
 exchange - vmm-handles  
 barrier()

Custom set access  
 register w/ IB NIC  
 ...

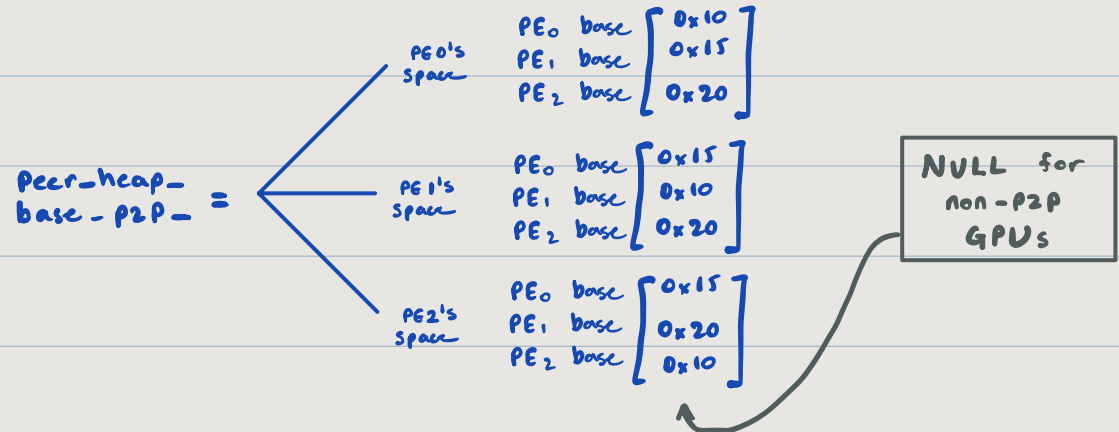
## FINAL GOAL: nvshmem-malloc (size)

- ① Intranode: establish a symmetric heap & a range of addrs. for each nvlink GPU

(A)



(B)



### Usage

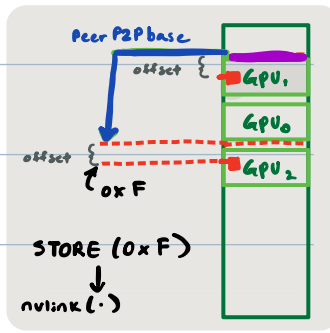
nvshmem-int-p(local sym heap, some value, target PE)

Put (some value) in (target PE) at address (sym heap address)

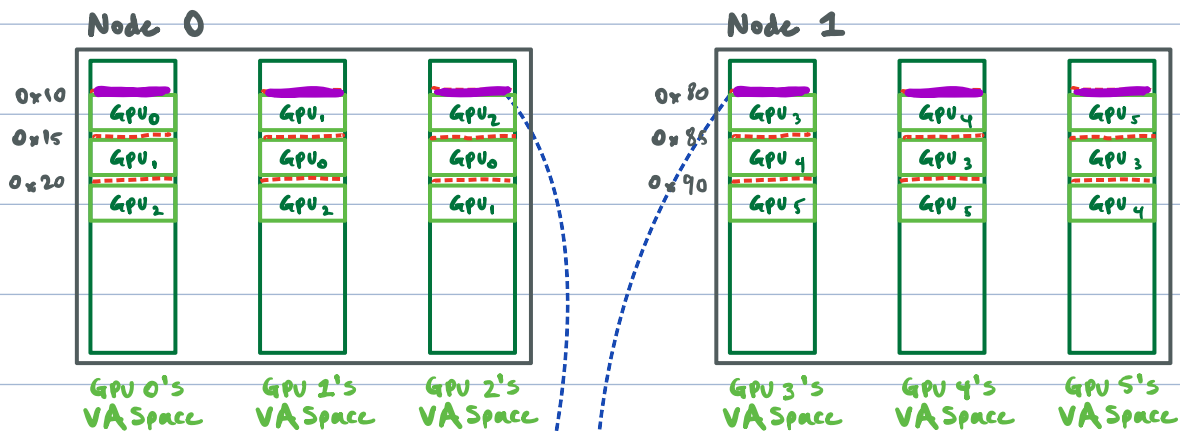
addr to place into = peerheapbaseP2P[target PE] + offset

VA that CUDA UVA will use to transport via NVLINK

local sym heap addr - local base addr = some local addr. in my local sym. heap!



② Every GPU sees global peer-heap-base-remote [ ]:



peer-heap-base-remote = 

|                 |                 |
|-----------------|-----------------|
| PE <sub>0</sub> | PE <sub>1</sub> |
|-----------------|-----------------|

= { 0x1000 — PE<sub>0</sub>  
0x1000 — PE<sub>1</sub>  
0x1000 — PE<sub>2</sub>  
0x3000 — PE<sub>3</sub>  
0x3000 — PE<sub>4</sub>  
0x3000 — PE<sub>5</sub> }

`nvshmem-int-p(Raddr, some value, targetPE)`

if not p2p:

`remote-base = peer-heap-base-remote[PE]`

`final-remote-addr = remote-base + offset`

`post-rdma-write-to-nic(Val, finaladdr, rkey)`

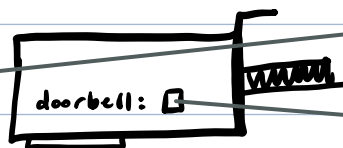
**GPAPI thread**

- writes WQE to buffer
- ring doorbell
- doorbell wakes NIC to send data

"IBGQA"

VRAM

WQE:

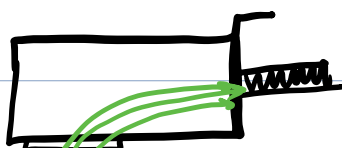


go to addr 0x795AE  
grab 4 bytes  
send to node X

GPU just writes manifest

doorbell: if 1, wake up & fulfill the WQE

presses doorbell



GPU transfers data  
CPU / kernel never touched

NIC handles flowing data:  
GPU continues